

Name: Prathamesh Arvind Jadhav

Roll No: 3014

EXPERIMENT NO: 9

Title:

Number Plate Detection and Recognition System

Aim:

To develop an end-to-end deep learning-based application that detects and recognizes vehicle number plates from images through a simple web UI.

Objectives:

1. Implement a YOLOv5-based object detection pipeline to localize number plates in input images.
 2. Integrate EasyOCR to perform optical character recognition on the cropped number plate regions.
 3. Build a Streamlit web interface to allow users to upload vehicle images and view detection results interactively.
 4. Ensure the system properly handles typical scenarios, including varying plate sizes, angles, and lighting conditions.
 5. Organize project artifacts, dependencies, and outputs using best practices (e.g., .gitignore, folder structure).
-

Theory:

1. Object Detection with YOLOv5

- YOLO (You Only Look Once) is a one-stage object detection algorithm that predicts bounding boxes and class probabilities in a single evaluation.
- YOLOv5 improves on earlier versions with architectural refinements in the backbone (Cross Stage Partial networks) and neck (Path Aggregation Network), leading to faster inference and higher accuracy.
- Inference involves passing an image through the model, obtaining predictions (results.), and filtering by confidence threshold to localize the plate region.

2. Optical Character Recognition (OCR) with EasyOCR

- OCR converts image text into machine-encoded text. EasyOCR leverages deep convolutional recurrent neural networks to handle multilingual text detection and recognition.
- After cropping the detected plate region, EasyOCR's `reader.readtext()` analyzes the image patch, outputs bounding boxes, recognized text, and confidence scores.

3. Streamlit UI

- Streamlit provides a declarative API for building data apps; `st.file_uploader` lets users upload images, `st.image` displays them, and control structures in `app.py` orchestrate the detection pipeline.
- Folder management (`uploads/`, `results/`) ensures persistence of input and output files, facilitating reproducibility and debugging.

4. Project Organization and Best Practices

- Dependencies are managed via `requirements.txt` and Python virtual environments (e.g., `.venv`).
 - `.gitignore` excludes temporary files, model artifacts (`*.pt`, YOLOv5 runs), upload/result directories, and environment folders to keep the repository clean.
-

Code:

```
import streamlit as st
```

```
import cv2
```

```
import torch
```

```
import numpy as np
```

```
import easyocr
```

Name: Prathamesh Arvind Jadhav

Roll No: 3014

```
import os
```

```
from PIL import Image
```

```
# Load YOLOv5 model
```

```
model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)
```

```
model.conf = 0.4 # confidence threshold
```

```
# Initialize EasyOCR reader
```

```
reader = easyocr.Reader(['en'])
```

```
# Create folders if they don't exist
```

```
os.makedirs("uploads", exist_ok=True)
```

```
os.makedirs("results", exist_ok=True)
```

```
st.title("❑ Number Plate Detection with OCR")
```

```
# File uploader
```

```
uploaded_file = st.file_uploader("Upload an image of a car",  
type=["jpg", "png", "jpeg"])
```

Name: Prathamesh Arvind Jadhav

Roll No: 3014

if uploaded_file is not None:

1. Save the uploaded image

upload_path = os.path.join("uploads", uploaded_file.name)

with open(upload_path, "wb") as f:

f.write(uploaded_file.getbuffer())

2. Display the uploaded image

st.image(upload_path, caption="Uploaded Image",
use_container_width=True)

3. Run detection

img = np.array(Image.open(upload_path).convert("RGB"))

results = model(img)

4. Render detections onto the image

annotated = results.render()[0] # returns a list; take first element

YOLOv5 renders in BGR, convert to RGB

annotated_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)

5. Save the annotated image to results/

Name: Prathamesh Arvind Jadhav

Roll No: 3014

```
result_path = os.path.join("results", uploaded_file.name)
```

```
Image.fromarray(annotated_rgb).save(result_path)
```

```
# 6. Display the annotated (detected) image
```

```
st.image(result_path, caption="Detected Plates",  
use_container_width=True)
```

```
# 7. Perform OCR on each detected bounding box
```

```
st.subheader("☐ Detected Plate Text:")
```

```
for i, det in enumerate(results.xyxy[0]):
```

```
    x1, y1, x2, y2 = det[:4].int().tolist()
```

```
    crop = img[y1:y2, x1:x2]
```

```
    if crop.size:
```

```
        ocr_results = reader.readtext(crop)
```

```
        for _, text, _ in ocr_results:
```

```
            st.success(f"Plate {i+1}: {text}")
```

```
st.info("☐ Done! Upload another image to detect more plates.")
```

Name: Prathamesh Arvind Jadhav
Roll No: 3014

Output:

Deploy ⋮

Number Plate Detection with OCR

Upload an image of a car



Drag and drop file here
Limit 200MB per file • JPG, PNG, JPEG

Browse files



Cars0.png 177.4KB



Uploaded Image



Detected Plates

Detected Plate Text:

Plate 1: car 0.421

Plate 1: KLCACA2555

✓ Done! Upload another image to detect more plates.

Name: Prathamesh Arvind Jadhav

Roll No: 3014

Conclusion:

This project demonstrates a streamlined workflow for vehicle number plate detection and recognition by combining state-of-the-art deep learning and OCR techniques within a user-friendly Streamlit interface. YOLOv5 effectively localizes license plates in diverse conditions, while EasyOCR accurately extracts alphanumeric text from the detected regions. The modular design—from image upload to result visualization—ensures ease of use and extensibility. By adhering to best practices for code organization, environment management, and version control, the solution is maintainable and ready for further enhancements, such as custom training on domain-specific datasets or integration into real-time video streams.